

ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE
FAKULTA STAVEBNÍ, OBOR GEODÉZIE A KARTOGRAFIE
KATEDRA GEOMATIKY

Algoritmy digitální kartografie a GIS

Geometrické vyhledávání bodu

Rok	Semestr	Skupina	Vypracovali	Datum
2025/2026	Letní		František Gurecký Adam Králič Vítek Veselý	15. 3. 2026

1 Zadání

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, \dots, P_n\}$, analyzovaný bod q .

Výstup: P_i , $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q .

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

2 Teoretická část

2.1 Ray Crossing Algorithm (RCA)

Základní princip pro detekci polohy bodu

Algoritmus Ray Crossing (metoda polopřímky) je založen na Jordanově větě o křivkách. Ta říká, že jednoduchá uzavřená křivka rozděluje rovinu na dvě disjunktní oblasti (vnitřek a vnějšek). Pro určení polohy analyzovaného bodu $q(x_q, y_q)$ vůči polygonu P je z tohoto bodu vedena fiktivní polopřímka libovolným směrem do nekonečna. V implementaci je pro zjednodušení výpočtů zvolen paprsek rovnoběžný s kladnou poloosou x .

Základní kroky algoritmu jsou následující:

1. **Redukce souřadnic:** Pro zjednodušení výpočtů se počátek souřadnicového systému posune do analyzovaného bodu q . Nové souřadnice vrcholů hrany polygonu p_i a p_{i+1} jsou vypočteny jako rozdíl vůči bodu q :

$$x'_i = x_i - x_q, \quad y'_i = y_i - y_q$$

2. **Filtrování hran:** Algoritmus iteruje přes všechny hrany polygonu. Aby polopřímka směřující vpravo mohla protnout hranu, musí koncové body hrany ležet v opačných polorovinách vůči ose x . Tato podmínka je testována porovnáním redukováných y -ových souřadnic.
3. **Výpočet průsečíku:** Pokud hrana splňuje výše uvedenou podmínku, je vypočtena x -ová souřadnice průsečíku x_m hrany s osou x (tedy s naším paprskem) pomocí lineární interpolace:

$$x_m = \frac{x'_{i+1}y'_i - x'_iy'_{i+1}}{y'_{i+1} - y'_i}$$

4. **Započítání průsečíku:** Pokud je vypočtený průsečík v kladné části osy x ($x_m > 0$), znamená to, že paprsek protnul hranu v požadovaném směru. V takovém případě se inkrementuje počítadlo průsečíků k .

5. **Vyhodnocení:** Na konci iterace se vyhodnotí parita počítadla k . Je-li celkový počet protnutí lichý ($k \pmod{2} = 1$), bod se nachází **uvnitř** polygonu. Je-li sudý, bod leží **vně**.

Ošetření singulárních případů

Při implementaci algoritmu je nutné ošetřit specifické geometrické situace, které by mohly vést k chybné detekci:

- **Paprsek procházející vrcholem polygonu:** Tato situace nastává, pokud y -ová souřadnice některého z vrcholů je přesně rovna y -ové souřadnici bodu q (redukováná souřadnice $y'_i = 0$). V implementaci je tento případ elegantně ošetřen použitím asymetrických relačních operátorů při filtrování hran (ostrá a neostrá nerovnost). Algoritmus hranu započítá pouze tehdy, platí-li:

$$(y'_{i+1} > 0 \wedge y'_i \leq 0) \vee (y'_i > 0 \wedge y'_{i+1} \leq 0)$$

Díky tomu se průchod vrcholem započítá vždy jen pro jednu ze dvou přilehlých hran, čímž nedojde k chybnému zdvojení průsečíku.

- **Bod ležící přímo na hraně polygonu:** Z matematického hlediska, pokud analyzovaný bod q leží přímo na obvodu polygonu, hodnota průsečíku x_m je rovna nule. Pro striktní zachycení tohoto stavu je vhodné provést test kolinearit (vektorový součin roven nule) a zkontrolovat, zda bod q spadá do hraničního boxu (min-max boxu) dané úsečky.

2.2 Winding Number Algorithm (WNA)

Základní princip pro analýzu polohy bodu

Algoritmus Winding Number (metoda úhlů) je založen na výpočtu celkového úhlu, pod kterým je polygon vidět z analyzovaného bodu q . Pokud se bod nachází uvnitř polygonu, součet orientovaných úhlů, které svírají jednotlivé hrany polygonu s bodem q , je roven 2π (nebo -2π v závislosti na orientaci polygonu). Pokud bod leží vně, součet úhlů se vyruší a výsledek je 0.

Postup výpočtu implementovaný v algoritmu se skládá z následujících kroků:

1. **Výpočet dílčího úhlu:** Pro každou hranu tvořenou vrcholy p_i a p_{i+1} jsou nejprve určeny vektory $\vec{u} = p_i - q$ a $\vec{v} = p_{i+1} - q$. Velikost neorientovaného úhlu θ je vypočtena pomocí skalárního součinu těchto vektorů a funkce arkus kosinus:

$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{|\vec{u}||\vec{v}|}\right)$$

Pro zabránění chybám způsobeným nepřesností v plovoucí řádové čárce je hodnota pro arkus kosinus před výpočtem omezena do intervalu $[-1.0, 1.0]$.

2. **Určení orientace (znaménka) úhlu:** Aby bylo možné úhly správně sčítat a odčítat, je nutné určit vzájemnou polohu bodu q vůči orientované úsečce $p_i p_{i+1}$. To se provádí pomocí

z -ové složky vektorového součinu směřujícího vektoru hrany a vektoru směřujícího z počátku hrany do bodu q :

$$w = (x_{i+1} - x_i)(y_q - y_i) - (y_{i+1} - y_i)(x_q - x_i)$$

Je-li $w > 0$, bod leží vlevo od hrany (úhel se přičítá). Je-li $w < 0$, bod leží vpravo (úhel se odčítá). V implementaci je kvůli numerické přesnosti výsledek porovnáván s malou tolerancí (*epsilon*).

3. **Vyhodnocení:** Orientované úhly ze všech hran se sečtou do celkové sumy Ω . Na závěr je absolutní hodnota této sumy porovnána s hodnotou 2π . Pokud je rozdíl menší než zvolená odchylka, bod je vyhodnocen jako ležící **uvnitř** polygonu.

Ošetření singulárního případu: bod ležící přímo na hraně

Specifickým případem je situace, kdy analyzovaný bod q leží přímo na některé z hran polygonu. V takovém případě by výpočet úhlu selhal (nebo vedl na neočekávané výsledky), protože velikost vektoru z bodu q k bodu na hraně by se blížila nule a vektorový součin pro určení orientace by byl nulový.

V implementaci je tento singulární případ elegantně podchycen a vyřešen ještě před samotným výpočtem úhlu:

- **Test kolinearity:** Pokud výsledek vektorového součinu w spadne do neutrálního pásma (jeho absolutní hodnota je menší než definovaná tolerance), znamená to, že body p_i , p_{i+1} a q leží na jedné přímce ($w \approx 0$).
- **Test obálky (Bounding Box):** Zjištění, že body leží na jedné přímce, ještě nezaručuje, že bod q leží na úsečce tvořící hranu (může ležet na přímce mimo úsečku). Proto je následně ověřeno, zda souřadnice bodu q spadají do obdélníkové obálky tvořené body p_i a p_{i+1} (tzn. x_q a y_q jsou v mezích minima a maxima souřadnic daných vrcholů).

Pokud jsou obě podmínky splněny současně, algoritmus okamžitě detekuje, že se bod nachází na obvodu polygonu, iteraci ukončí a vyhodnotí bod jako ležící **uvnitř** polygonu.

2.3 Společné singulární případy

Ošetření situace, kdy je bod q totožný s vrcholem polygonu

V topologicky korektní (souvislé) polygonové mapě může nastat specifický případ, kdy analyzovaný bod q leží přesně na vrcholu, který může sdílet i více sousedících polygonů současně. Tento stav představuje kritickou singularitu pro oba testovací algoritmy, protože vzdálenost analyzovaného bodu q od dotčeného vrcholu p_i je v takovém případě nulová.

- **Vliv na Ray Crossing Algorithm:** Pokud je bod q totožný s vrcholem, počátek testovacího paprsku leží přímo na obvodu polygonu. Výpočet průsečíku x_m by bez explicitního ošetření vedl k nulové hodnotě ($x_m = 0$) a bod by mohl být chybně vyhodnocen jako ležící vně polygonu.

- **Vliv na Winding Number Algorithm:** Výpočet úhlu v tomto algoritmu striktně vyžaduje určení délky vektorů z bodu q k vrcholům hrany. Pokud $q \equiv p_i$, délka jednoho z vektorů je prokazatelně nulová ($|\vec{u}| = 0$ nebo $|\vec{v}| = 0$). Následný výpočet jmenovatele pro funkci arkus kosinus by bez ošetření vedl k fatální chybě dělení nulou.

Způsob ošetření v implementaci

Pro bezpečné podchycení tohoto stavu a splnění zadání je v implementaci využito následujících mechanismů:

1. **Prevence matematické chyby (WNA):** Před samotným výpočtem úhlu je explicitně kontrolována Euklidovská délka obou vektorů. Pokud je velikost alespoň jednoho z nich nulová, je výpočet přerušen a dílčí úhel je bezpečně vyhodnocen jako 0.0.
2. **Absorpce do testu kolinearitu:** Ve většině případů je tento stav u Winding Number algoritmu zachycen ještě dříve pomocí testu na kolinearitu bodu a hrany. Protože nulový vektor generuje nulový vektorový součin, bod je okamžitě zachycen kontrolou obálky hrany (Bounding Box hrany) a algoritmus vrací hodnotu **True**. Explicitní rovnost souřadnic bodu q a vrcholu p_i je nejrobustnějším způsobem, jak tento stav ošetřit i v Ray Crossing algoritmu.
3. **Zvýraznění vícenásobné incidence:** Vzhledem k tomu, že algoritmus iteruje přes všechny dostupné polygony (filtrováné pomocí min-max boxu), je detekce spuštěna nezávisle pro každý polygon. Pokud sdílí daný vrchol více polygonů v mapě, algoritmus vyhodnotí bod q jako incidující pro všechny tyto polygony. Díky struktuře grafického rozhraní a uchovávání výsledků v poli `__result` jsou následně všechny tyto polygony graficky zvýrazněny.

2.4 Optimalizace hledání: Využití min-max boxu

Princip rychlého vyřazení (Bounding Box)

Testování polohy bodu vůči polygonu pomocí algoritmů Ray Crossing nebo Winding Number je pro složité polygony výpočetně náročné. Pro zefektivnění celého procesu je před samotnou analýzou implementován krok rychlého předvýběru potenciálních polygonů. Algoritmus iteruje přes všechny dostupné polygony, které jsou nejprve filtrovány pomocí min-max boxu.

Min-max box (obálka polygonu) představuje nejmenší možný osově zarovnaný obdélník, do kterého lze daný polygon vepsat. Je definován dvěma body: minimálními a maximálními souřadnicemi vrcholů polygonu v rámci daného souřadnicového systému.

Postup vyhledávání potenciálních polygonů

Před spuštěním hlavních algoritmů je proveden rychlý plošný test:

1. Pro každý polygon se v konstantním čase ověří, zda souřadnice analyzovaného bodu $q(x_q, y_q)$ spadají do rozsahu jeho min-max boxu. Testuje se následující podmínka:

$$(x_q \geq x_{min}) \wedge (x_q \leq x_{max}) \wedge (y_q \geq y_{min}) \wedge (y_q \leq y_{max})$$

2. Pokud bod q této podmínce nevyhovuje, s naprostou jistotou se nemůže nacházet ani uvnitř samotného polygonu. Takový polygon je z dalšího výpočtu okamžitě vyřazen.
3. Pokud bod do min-max boxu spadá, polygon je označen jako potenciální kandidát a zařazen do vyfiltrovaného seznamu.

Díky této optimalizaci se výpočetně složitější testy (RCA a WNA) spouštějí pouze nad zlomkem polygonů, u kterých existuje reálná šance na incidenci s analyzovaným bodem. To vede k výraznému zrychlení běhu programu, což je naprosto klíčové zejména při práci s rozsáhlými mapovými podklady složenými z tisíců polygonů (např. při načítání a analýze dat z formátu SHP).

2.5 Rozšířené funkce: Řešení pro polygony s dírami

Geometrická definice a princip

Polygon s dírami (vícenásobně souvislá oblast) je geometrický útvar, který se skládá z jedné vnější hranice (obálky) a jedné či více vnitřních hranic (děr). Pro správné vyhodnocení topologické polohy bodu q je nutné zajistit, aby bod ležící uvnitř díry byl algoritmem korektně vyhodnocen jako ležící **vně** samotného polygonu.

Díky matematické povaze obou použitých algoritmů lze tuto problematiku řešit velmi elegantně bez nutnosti zavádět složité speciální testy, a to pouhým zahrnutím všech hran (vnějších i vnitřních) do výpočtu.

Aplikace pro Ray Crossing Algorithm (RCA)

Algoritmus paprsku zvládá polygony s dírami zcela přirozeně na základě vlastností liché a sudé parity:

- Pokud analyzovaný bod q leží ve vlastní ploše polygonu (mezi vnější a vnitřní hranicí), testovací paprsek protne vnější obvod (případně další hranice) tak, že celkový počet průsečíků k bude **lichý** ($k \pmod{2} = 1$). Bod je vyhodnocen jako **uvnitř**.
- Pokud bod q leží uvnitř díry, paprsek vedený z tohoto bodu musí nevyhnutelně protnout nejprve hranici díry a následně i hranici vnější obálky polygonu. Tím dojde ke dvěma průsečíkům (nebo obecně **sudému** počtu). Sudá parita ($k \pmod{2} = 0$) pak algoritmu správně indikuje, že bod leží **vně** polygonu.

Aplikace pro Winding Number Algorithm (WNA)

U metody úhlů je klíčem k řešení děr správná *orientace* vrcholů polygonu, která typicky vychází z formátu dodaných dat (např. standard definovaný pro ESRI Shapefile).

- Vnější hranice polygonu a vnitřní hranice (díry) jsou orientovány vzájemně opačně (typicky vnější hranice po směru hodinových ručiček a díry proti směru, nebo naopak).
- Při výpočtu sumy úhlů Ω přes všechny části polygonu (vnější i vnitřní hrany) se příspěvky sčítají i se svými znaménky.

- Pokud bod q leží v ploše polygonu, vnější hranice vygeneruje úhel $\pm 2\pi$, zatímco díry k sumě nepřispějí (bod leží vně děr, jejichž úhly se vyruší na 0). Výsledek je $|\Omega| = 2\pi$ (**uvnitř**).
- Pokud bod q leží přímo uvnitř díry, vnější hranice vygeneruje úhel např. $+2\pi$ a opačně orientovaná hranice díry vygeneruje úhel -2π . Tyto hodnoty se vzájemně odečtou:

$$\Omega = 2\pi + (-2\pi) = 0$$

Výsledný nulový úhel algoritmu zcela korektně potvrdí, že bod se nenachází v ploše polygonu, ale **vně**.

Pro úspěšnou implementaci tohoto rozšíření je primárně nutné přizpůsobit datový model tak, aby sdružoval vnější a vnitřní hranice do jednoho celku (případně strukturovat načítání částí ze SHP souboru) a zpracovával jejich hrany v rámci jediné iterace testu.

3 Datový model a struktury

3.1 Datová reprezentace: Špagetový model

Pro uložení polygonové mapy v operační paměti byl v souladu se zadáním využit tzv. špagetový datový model. Jedná se o základní vektorový model, ve kterém není udržována žádná explicitní topologie (tedy informace o vzájemné sousednosti, sdílených hranách či uzlech). Každý polygon je v paměti reprezentován jako zcela nezávislý a samostatný objekt, který je plně definován vlastním uspořádaným seznamem vrcholů (geometrických souřadnic). Důsledkem tohoto přístupu je, že hrany a vrcholy, které sdílí více sousedících polygonů, jsou v paměti uloženy duplicitně pro každý polygon zvlášť.

V rámci samotné implementace v jazyce Python je tento model realizován pomocí vlastní třídy `Polygon`. Tato třída efektivně dědí z vestavěné třídy `QPolygonF` (součást frameworku `PyQt6`), což umožňuje snadnou integraci s grafickým uživatelským rozhraním a přímé využití vestavěných metod pro práci s polygonovou geometrií.

Pro potřeby algoritmů a vizualizace je třída `Polygon` rozšířena o následující dodatečné atributy:

- **id**: Celočíselný identifikátor polygonu, který slouží ke snadnému rozlišení entit a formátovanému výpisu výsledků v uživatelském rozhraní.
- **bbox**: Atribut uchovávající obdélníkovou obálku (min-max box) polygonu. Je definován polem dvou bodů a slouží pro rychlou prostorovou indexaci a filtraci (preselection) před spuštěním samotných analytických algoritmů.

Jednotlivé vrcholy jsou do struktury polygonu postupně načítány a ukládány pomocí vytvořené metody `addVertex`. V hlavní třídě aplikace (`Draw`) je pak celá polygonová mapa udržována v jednoduchém jednorozměrném poli `__pol`, které iterativně procházejí analytické i vykreslovací funkce.

3.2 Načítání a zpracování dat

Zpracování vlastního textového formátu

Základním požadavkem zadání bylo načítání polygonů z textového souboru ve vlastním formátu. Pro tyto účely lze data strukturovat jako prostý textový soubor, kde je každý polygon uvozen svým unikátním identifikátorem, po kterém na dalších řádcích následují uspořádané dvojice souřadnic (x, y) jednotlivých vrcholů oddělené mezerou či tabulátorem. V rámci finální implementace je však – vedle možnosti interaktivního zadávání vrcholů myši přímo do grafického plátna – kladen hlavní důraz na mnohem robustnější a standardizované načítání geografických dat z formátu SHP.

Načítání a parsování dat z formátu SHP (Shapefile)

Pro efektivní načítání reálných geografických a prostorových dat byl implementován parser souborů typu Shapefile (SHP). Tento krok umožňuje snadné testování na komplexních a nekonvexních polygonech s mnoha vrcholy (např. mapy krajů či okresů). Zpracování je realizováno s využitím externí pythonovské knihovny `shapefile` (PyShp) a probíhá v následujících logických krocích:

1. **Otevření souboru a načtení tvarů:** Skrze grafické rozhraní a systémový dialog `QFileDialog` uživatel vybere zdrojový soubor s příponou `.shp`. Knihovna soubor načte a vytvoří iterátor přes všechny dostupné geometrické tvary (`shapes`) v dané mapě.
2. **Extrakce a konverze obálek (Bounding Box):** U každého načítaného tvaru se nejprve vyčte jeho hraniční obálka (`shape.bbox`). Ta je pomocí naimplementované metody `bboxToQPoint` převedena na dvojici bodů typu `QPointF`, které rovnou tvoří min-max box pro optimalizaci vyhledávání.
3. **Transformace souřadnicového systému:** Geografická data (jako např. systém S-JTSK) typicky využívají kartézský systém, kde osa y roste směrem nahoru. Vykreslovací plátno (Canvas) ve frameworku PyQt6 má však počátek $(0, 0)$ v levém horním rohu a osa y směřuje dolů. Při parsování bodů ze seznamu `shape.points` je proto nezbytné u všech y -ových souřadnic invertovat znaménko.
4. **Aplikace posunu (Offset):** Souřadnice v reálných geografických systémech nabývají absolutních hodnot v řádech stovek tisíců až milionů, což by bez úprav znemožnilo jejich zobrazení na standardní obrazovce. Během načítání vrcholů je proto aplikován předem definovaný konstantní posun (tzv. *offset*, v implementaci konkrétně upraveno o $+675\,000$ pro osu x a $-1\,100\,000$ pro osu y). Tento krok data normalizuje a přesune do viditelné a snadno manipulovatelné oblasti plátna.
5. **Uložení do datové struktury:** Transformované vrcholy jsou metodou `addVertex` postupně přidávány do nově vytvořené instance vlastní třídy `Polygon`. Tomuto objektu je rovněž přiřazeno identifikační číslo (`id`) a transformovaný Bounding Box. Hotový polygon je následně vložen do hlavního seznamu `__pol`.

3.3 Zdroj dat

Zadání úlohy umožňovalo pro testování algoritmů nad nekonvexními polygony navrhnout buď vlastní generátor, nebo využít existující geografická data. Pro důkladné prověření implementace byla zvolena druhá možnost, tedy využití reálných prostorových dat.

Jako hlavní testovací sady byly použity vektorové mapové podklady ve formátu Shapefile (SHP), které reprezentují administrativní členění České republiky. Konkrétně jsou pro testování využívány:

- **Mapy krajů,**
- **Mapy okresů,**
- **Mapy obcí.**

Využití těchto existujících geografických dat přináší zásadní výhodu pro ověření robustnosti celého programu. Hranice reálných územních celků tvoří vysoce komplexní, nekonvexní polygony, které přirozeně obsahují ostré úhly, sdílené úseky hranic a společné uzly pro více polygonů současně. Taková struktura představuje ideální zátěžový test pro Ray Crossing i Winding Number algoritmus a dokonale na ní lze demonstrovat i ošetření vícenásobné incidence (bod na pomezí tří obcí). Zároveň se na velkém počtu polygonů (např. u mapy obcí) v praxi prokáže obrovská časová úspora díky předvýběru pomocí min-max boxů.

Pro usnadnění uživatelské práce je v aplikaci naimplementována funkce, která při načtení specifických vzorových dat automaticky přizpůsobí parametry zobrazení (nastaví vhodný *zoom* a *pan*). Díky tomu je načtená mapa ihned vycentrovaná na plátně a připravená k vizuální i geometrické analýze.

4 Návrh uživatelského rozhraní a implementace

4.1 Využití grafického frameworku QT

Architektura uživatelského rozhraní je logicky rozdělena do dvou hlavních komponent, které oddělují samotné rozvržení ovládacích prvků od vykreslovací a aplikační logiky:

1. Hlavní okno aplikace (`MainForm.py`)

Struktura hlavního okna byla pro efektivitu navržena pomocí nástroje Qt Designer a následně vygenerována do Python kódu (pomocí `pyuic6`). Základem je třída `QMainWindow`, která sdružuje následující prvky:

- **Ovládací prvky:** Okno obsahuje standardní nabídkový pruh (`QMenuBar`) a nástrojovou lištu (`QToolBar`) pro rychlý přístup k akcím (načtení souboru, přepínání režimu vstupu, spuštění algoritmů RCA a WNA, vyčištění plátna). Jednotlivé akce (`QAction`) jsou pomocí signálů propojeny s příslušnými metodami vykreslovacího plátna.

- **Rozdělovač (QSplitter):** Centrální widget je vertikálně rozdělen pomocí komponenty `QSplitter`. Ta umožňuje uživateli dynamicky měnit poměr velikosti mezi horním grafickým plátnem a spodním textovým panelem.
- **Textový výstup (Log):** Ve spodní části okna se nachází textové pole (`QPlainTextEdit`) nastavené pouze pro čtení. Slouží jako konzole pro logování událostí s časovým razítkem (načtení souboru, zvolené metody, počty bounding boxů) a pro textový výpis finálních výsledků analýzy.

2. Interaktivní grafické plátno (draw.py)

Jádro vizualizace a interakce tvoří vlastní třída `Draw`, která dědí ze základního prvku `QWidget`. Tato třída spravuje datový model (pole polygonů a analyzovaný bod q), provolává analytické algoritmy a stará se o překreslování obrazovky. Pro zajištění interaktivity jsou v ní přetíženy následující událostní metody (*Event Handlers*) frameworku Qt:

- **paintEvent:** Metoda pro samotné vykreslování geometrie. Využívá třídu `QPainter` pro kreslení polygonů a elipsy (bodu). Důležitou součástí je implementace transformační matice (`QTransform`), která aplikuje aktuální měřítko (*zoom*) a posun (*pan*) na všechny vykreslované objekty.
- **mousePressEvent:** Zpracovává kliknutí levým tlačítkem myši. Podle aktuálního stavu aplikace (`__add_vertex`) se transformované souřadnice kliknutí uloží buď jako nový vrchol kresleného polygonu, nebo jako nová poloha analyzovaného bodu q .
- **wheelEvent** a **keyPressEvent:** Slouží k plynulému přibližování a oddalování plátna pomocí kolečka myši a k posunu mapy (panning) pomocí směrových šipek na klávesnici.

4.2 Grafická prezentace výsledků

Způsob vizuálního zvýraznění nalezeného polygonu

Zadání úlohy požaduje, aby byl nalezený (incidující) polygon graficky zvýrazněn vhodným způsobem, například vyplněním, šrafováním nebo blikáním. V rámci implementace ve frameworku PyQt6 byl zvolen přístup dynamické změny barvy výplně a obrysu.

O samotné vykreslování se stará přetížená metoda `paintEvent` za využití objektu `QPainter`. Vykreslovací logika rozlišuje dva stavy polygonu:

- **Základní stav:** Polygony, které neobsahují analyzovaný bod, jsou vykreslovány standardním stylem – žlutou výplní (`Qt.GlobalColor.yellow` v objektu `QBrush`) a červeným obrysem (`Qt.GlobalColor.red` v objektu `QPen`).
- **Zvýrazněný stav:** Pokud je polygon identifikován analytickým algoritmem jako incidující, je jeho reference uložena do interního seznamu výsledků (`__result`). Při následném překreslování (`repaint`) plátna je pro takový polygon použit inverzní styl – plná červená výplň se žlutým obrysem. Tato změna barvy plochy tvoří výrazný vizuální kontrast a uživateli okamžitě a zřetelně indikuje výsledek prostorového dotazu.

Zvýraznění všech polygonů při výskytu singulárních případů

Kritickým požadavkem úlohy je správná grafická interpretace singulárních případů, typicky situace, kdy analyzovaný bod leží na sdílené hraně nebo je totožný s vrcholem, který sdílí více sousedících polygonů v mapě. V takovém případě musí být graficky zvýrazněny všechny tyto polygony současně.

Tento požadavek je v architektuře aplikace splněn díky návrhu hlavní analytické smyčky (metoda `analyzer`):

1. Výpočet se neukončuje po nalezení prvního incidujícího polygonu. Namísto toho algoritmus (RCA i WNA) vždy iteruje přes všechny dostupné polygony, které prošly primárním filtrem min-max boxu.
2. Pokud algoritmus vrátí pro daný polygon pozitivní shodu (`True`), je polygon přidán metodou `append` do seznamu výsledků `__result`.
3. V případě, že se bod q nachází na hranici či vrcholu dvou a více polygonů, algoritmus vyhodnotí incidenci pozitivně pro každý z nich nezávisle na sobě. Pole `__result` tak po dokončení smyčky obsahuje reference na všechny zasažené polygony.
4. Následné volání `paintEvent` aplikuje zvýrazňovací grafický styl na všechny položky přítomné v seznamu `__result`, čímž dojde k bezpečnému a přehlednému zvýraznění vícenásobné incidence přímo na mapě. Obdobně je tento stav propsán i do textového logu, kde jsou vypsána ID všech dotčených polygonů.

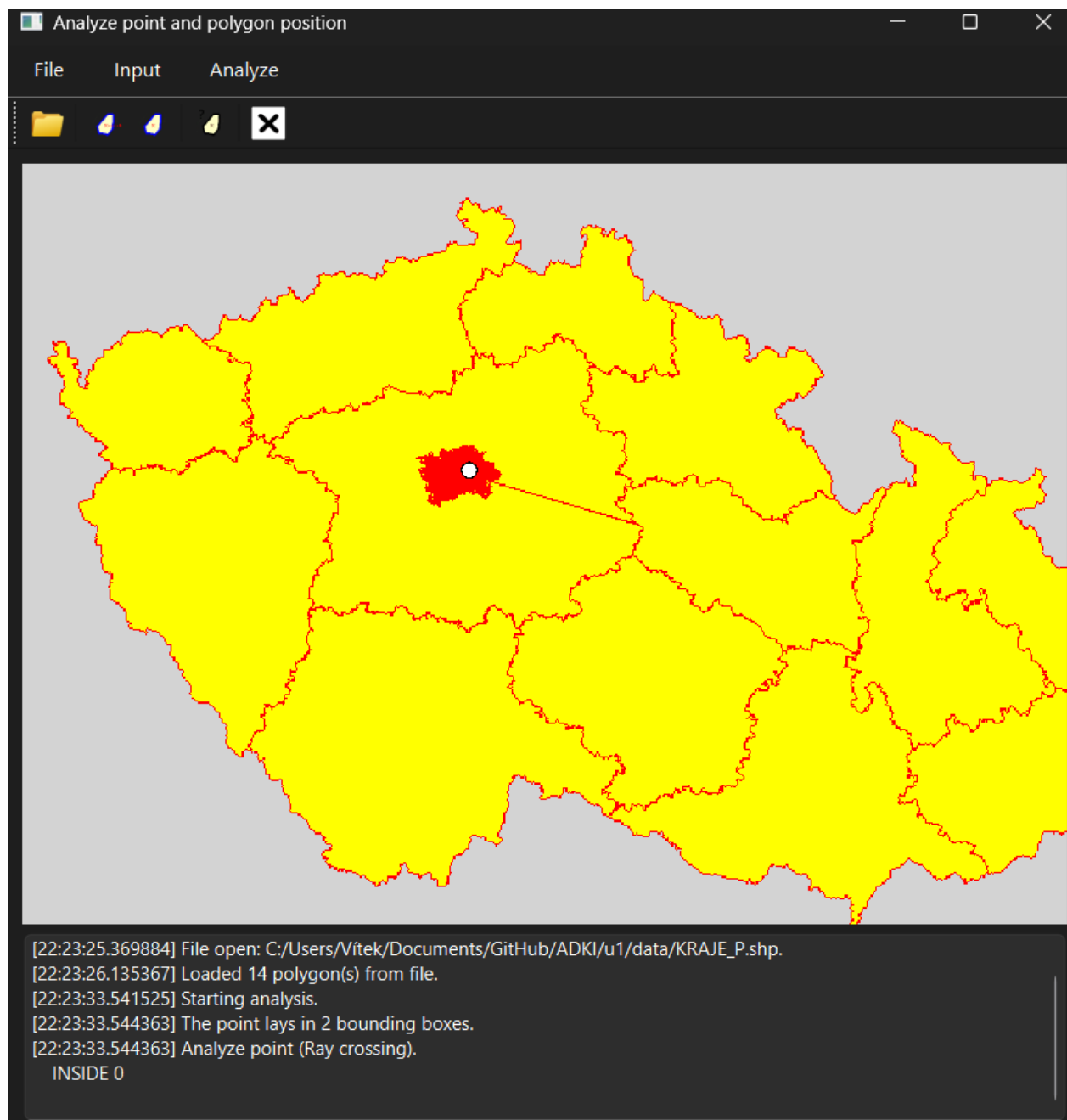
5 Testování a dosažené výsledky

Pro ověření správnosti implementovaných algoritmů, robustnosti datových struktur a celkové stability aplikace byla provedena série systematických testů. Testování probíhalo jak na manuálně zadávaných polygonech (přímo v grafickém plátně), tak na rozsáhlých reálných geografických datech načtených z formátu SHP.

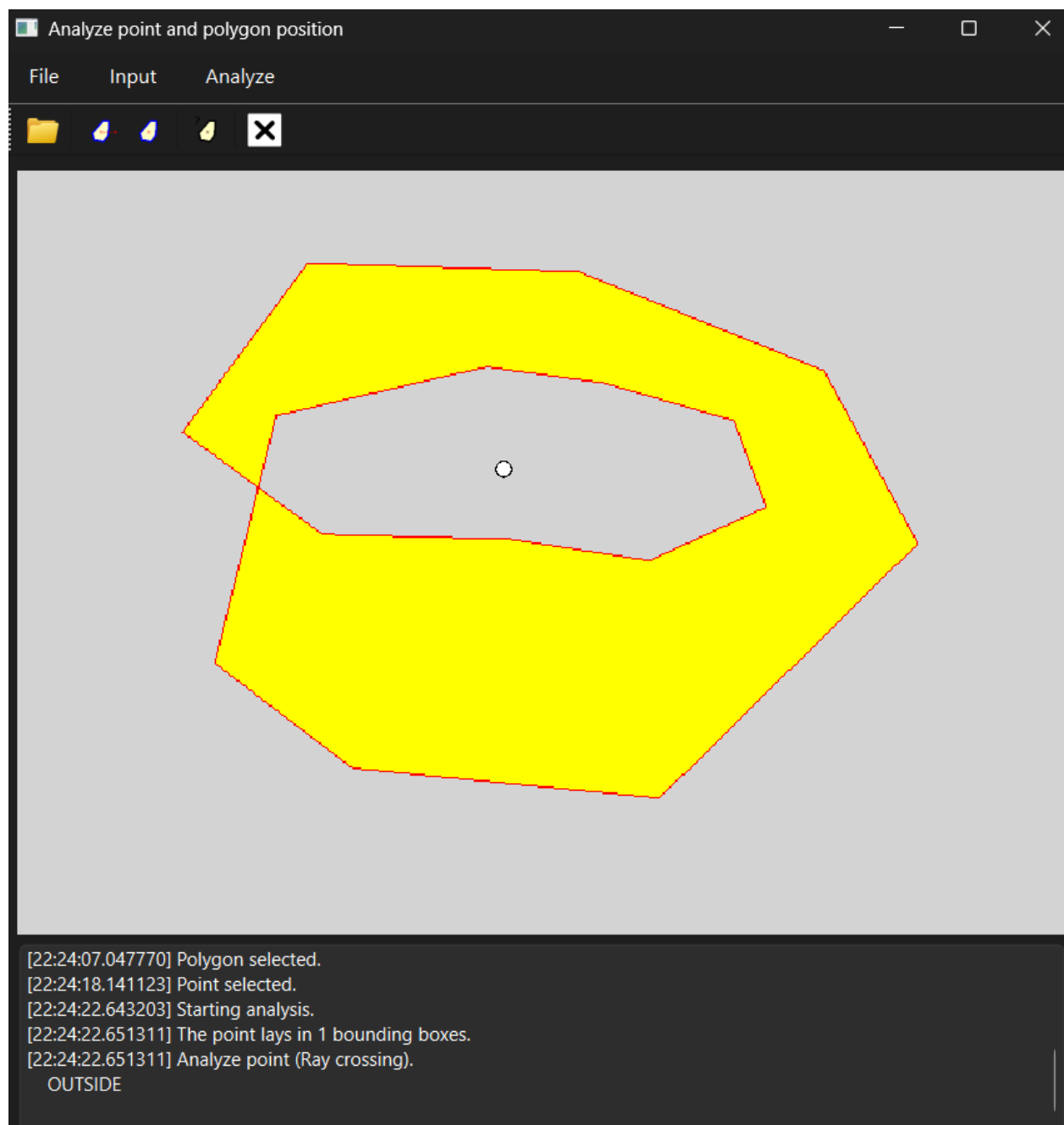
5.1 Testování na různých typech polygonů

- **Konvexní polygony:** Základní testy na jednoduchých konvexních tvarech (trojúhelníky, obdélníky) proběhly bez problémů. Oba algoritmy (RCA i WNA) vracely shodné a stoprocentně přesné výsledky v reálném čase.
- **Nekonvexní polygony:** Testování na složitých nekonvexních tvarech (např. spletené hranice) potvrdilo teoretické předpoklady. Paprsek v algoritmu RCA správně zaznamenával vícenásobná protnutí (střídání stavu vnitřek/vnějšek), přičemž konečná parita vždy odpovídala skutečné poloze bodu. WNA naproti tomu spolehlivě přičítal a odečítal orientované úhly tak, aby výsledná suma dala 2π pro vnitřek a 0 pro vnějšek.

- **Polygony s dírami:** Při testování na datech obsahujících díry (vnitřní hranice orientované opačně k vnější obálce) program správně vyhodnotil bod ležící uvnitř této díry jako ležící **vně** polygonu. U RCA byla výsledkem sudá parita průsečíků, u WNA se úhel obálky a úhel díry vzájemně vyrušily na 0.



Obrázek 1: Ukázka nekonvexního polygonu: Bod se nachází pouze ve vnitřním polygonu.

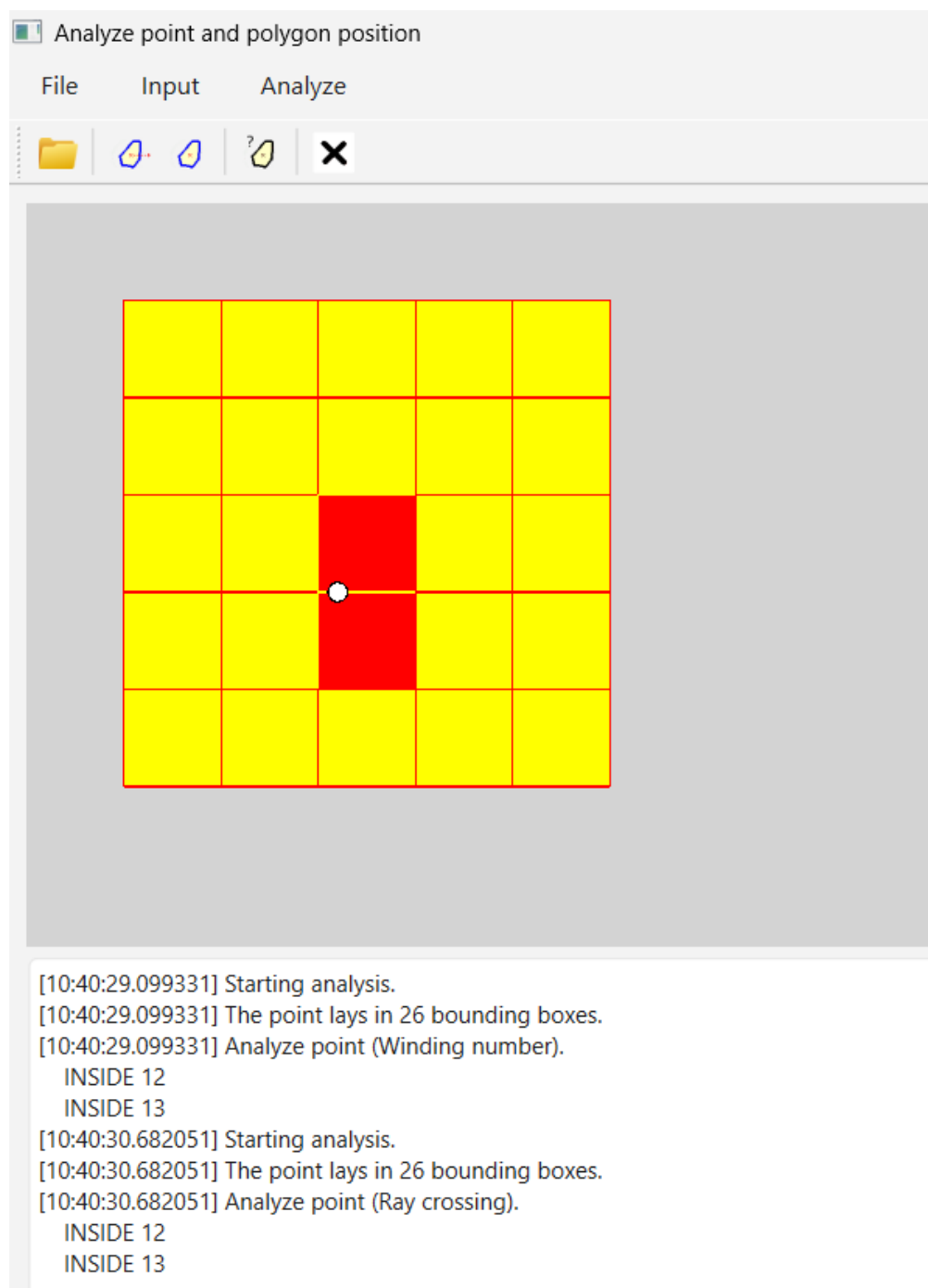


Obrázek 2: Ukázka nekonvexního polygonu: Bod neleží v polygonu.

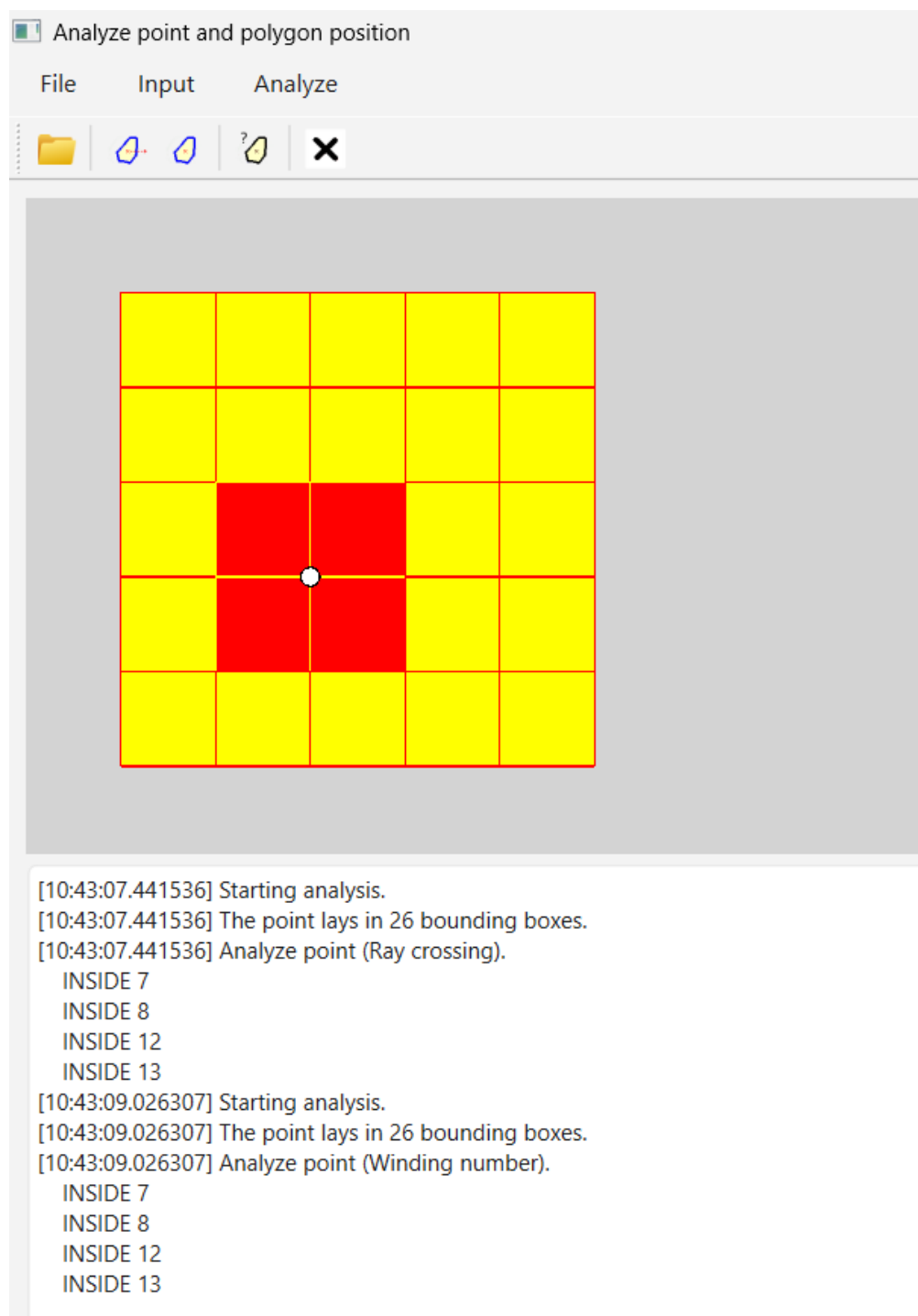
5.2 Testování hraničních a singulárních případů

Největší důraz byl kladen na testování geometrických singularit, které v praxi způsobují pády programů nebo chybné vyhodnocení:

- **Bod ležící přímo na hraně:** Algoritmus WNA prokázal vysokou spolehlivost díky explicitnímu testu na kolinearitě (vektorový součin blížící se nule s ohledem na *epsilon*) a následnému testu min-max obálky hrany. Bod na hraně byl okamžitě a bezpečně vyhodnocen jako incidující.
- **Bod totožný s vrcholem (vícenásobná incidence):** Při umístění analyzovaného bodu q přesně na sdílený vrchol více polygonů (např. trojmezí tří okresů) nedošlo k chybě dělení nulou. Metoda `calculateAngle` ve WNA tento stav podchytila přes Euklidovskou délku vektorů. Aplikace správně vyhodnotila incidenci pro všechny dotčené polygony a následně je všechny graficky zvýraznila červenou výplní.
- **Paprsek procházející vrcholem (RCA):** Při umístění bodu q tak, aby z něj vyslaný paprsek (rovnoběžný s osou x) přesně protínal některý z vrcholů polygonu, se potvrdila funkčnost navržených asymetrických relačních operátorů pro y -ové souřadnice. Nedocházelo ke zdvojenému započítání průsečíku u dvou sbíhajících se hran a parita zůstala korektní.



Obrázek 3: Testování singulárního případu: Bod ležící přímo na sdílené hraně dvou polygonů.



Obrázek 4: Testování singulárního případu: Bod totožný se sdíleným vrcholem čtyř polygonů.

6 Závěr

Předložená aplikace plně a úspěšně implementuje všechny stanovené požadavky zadání. Ústředním prvkem je funkční grafické uživatelské rozhraní postavené na frameworku PyQt6, které umožňuje interaktivní i dávkovou analýzu polohy bodu vůči souvislé polygonové mapě.

Byl naimplementován jak primárně požadovaný Ray Crossing Algorithm, tak i dodatečně hodnocený Winding Number Algorithm. U obou výpočetních metod byla věnována patřičná pozornost korektnímu ošetření topologických a matematických singularit – ať už se jednalo o průchod testovacího paprsku vrcholem, bod ležící přímo na hraně polygonu, nebo kritickou vícenásobnou incidenci bodu na sdíleném vrcholu. Podařilo se také plně realizovat požadavek na grafické zvýraznění nalezeného polygonu, a to včetně současného zvýraznění vícero entit při dotazu přímo na sdílený uzel.

Z datového hlediska aplikace spoléhá na požadovaný špagetový model. Z hlediska požadovaných nadstaveb program splňuje všechny parametry: provádí rychlý předvýběr kandidátů pomocí obdélníkových obálek (min-max box), matematicky podporuje polygony s dírami a umožňuje načítání reálných geografických dat (kraje, okresy, obce) ze standardizovaného souborového formátu SHP.

V Praze dne 15. 3. 2026

František Gurecký

Adam Králič

Vítek Veselý